AI-Powered Escape Room: Project Report

Course: Deep Learning, Mini Project 01

1. Objective

The goal was an interactive escape-room game in which a Large Language Model (Google Gemini) acts as the Game Master. The player is locked in a library and must explore objects, find clues, solve puzzles and enter a four-digit code to escape. Two versions were built. Version 1 lets the LLM run the whole game. Version 2 makes the game reliable by moving the rules, state and solutions into Python, so that the LLM only describes outcomes.

2. Game design

The room has seven interactive objects: a bookshelf, a desk, a locked drawer, an oil painting, a wall box behind the painting, a stopped grandfather clock and the exit door. There are three chained puzzles:

- **Puzzle 1:** four oddly placed books carry the numbers 4, 1, 7, 3. Entering 4173 opens the drawer, which holds a brass key.
- **Puzzle 2:** the brass key opens the wall box behind the painting. It contains a note about the hour the great clock stopped.
- **Puzzle 3:** the clock shows 9:47. The note says hour first, then minutes, so the door code is 0947.

In Version 2 the door keypad has no power until the wall box is opened, which forces the puzzles to be solved in order.

3. Version 1: LLM as Game Master

Version 1 is a Streamlit chat application (app_v1.py) that calls the Gemini API through the google-genai library. A system prompt describes the room, the puzzles, the solutions and the rules. On every turn the full chat history is sent to Gemini, which is how the game remembers earlier actions. Gemini adds hidden tags to its replies: [ITEM: name] when the player gets an item and [ESCAPED] when the correct door code is entered. The application strips the tags, updates the inventory in the sidebar and shows a success message on escape. The sidebar also contains instructions and a Restart button.

This design is simple and engaging, but every decision depends on the LLM. The solutions are written in the prompt, the chat history is the only memory, and the inventory depends on the model remembering to add a tag.

4. Version 2: reliable game with Python-controlled state

Version 2 (app_v2.py) processes each player message in three steps. The architecture diagrams show the components of both versions.

- **Intent parsing.** Python rules first catch administrator or override claims, requests for secrets and claims of earlier progress. Otherwise Gemini converts the text into a small JSON action (examine, take, use, open, enter code, and so on). A keyword parser takes over if Gemini fails. Codes are extracted by Python, never by the LLM.
- **Validation and state.** A GAME_CONFIG dictionary stores the objects, items, note text and solutions outside any prompt. A state dictionary stores the inventory, discovered objects, flags such as drawer_open and door_unlocked, solved puzzles and failed attempts. Functions such as act_use, act_take and act_enter_code check the rules and return a structured result (success, reason, facts). Only these functions change the state.
- **Narration.** Gemini receives only the validated result and describes it in a few sentences. An output check rejects any reply that contains a code the player has not earned or that leaves out a required fact, and the plain facts from Python are shown instead. Gemini calls are retried on temporary overload (503), and the game still works with plain text if Gemini is unavailable.

A developer view in the sidebar shows the current state and the parsed action of the last turn, which makes the behaviour easy to inspect.

5. Testing

Five adversarial cases were designed (full table and screenshots in the separate test-case document): using a key that was not collected, entering the door code at the start, claiming that a puzzle was already solved, claiming to be the game administrator, and asking the Game Master to reveal all codes.

- **Version 2, live tests:** using the brass key without having it was refused (item_not_in_inventory), and entering 0947 at the start failed because the keypad had no power. The state did not change in either case.
- **Version 2, engine tests:** the Python rules were also run offline for all five cases and for a complete honest playthrough. The false claim, the administrator request and the request for secrets were refused with no state change, and the honest playthrough ended in an escape.

6. Problems encountered

- **Streamlit does not run in Jupyter.** Running the code in notebook cells caused missing ScriptRunContext warnings and a syntax error. The apps are now Python files started with streamlit run.
- **API key in the code.** An early notebook contained the key in plain text. The apps now read it from .streamlit/secrets.toml or an environment variable.
- **Changing model names and overload.** gemini-1.5-flash had been retired and gemini-2.5-flash was no longer available to new users, which returned 404 errors. The model name is now a single constant at the top of each file. A 503 overload error led to the retry logic and the fallbacks in Version 2.
- **The LLM as rule keeper.** The main design problem: an LLM can forget actions, invent objects, reveal clues early or accept false claims. This led to the split between Python (decides) and Gemini (narrates).

7. Improvements and conclusion

Compared with Version 1, Version 2 keeps a structured state, validates every action in Python, only lets the player use collected items, stores the solutions outside the LLM, remembers solved puzzles consistently, and handles invalid actions and adversarial messages without changing the state. The LLM is still used for what it does best: understanding free text and writing atmospheric descriptions.

Limitations remain. The keyword fallback understands a limited vocabulary, the rules for catching manipulation are pattern based, the game has a single room, and there is no automated test suite or saved progress. These would be the next steps.